

Pauta I1 2025-1

1. Preguntas de desarrollo (4 ptos)

1.a) (4 ptos) Dados los siguientes atributos de calidad: Seguridad, Disponibilidad, Escalabilidad, Confiabilidad, Performance. Arme 2 pares que más les acomoden entre estos NFR e indique, para cada par, un caso donde se pueda ver que uno se ve beneficiado por usar el otro, y otro caso donde uno se vea afectado negativamente porque se está usando el otro.

Respuesta: Depende de los casos que indique el alumno, debe ser claro el caso para cada par.

Ejemplo de respuesta:

- Seguridad – Disponibilidad:
 - Ventaja: Sistema de autorización en API Gateway de request permite que el backend no reciba spam no autorizado y poder mantenerse disponible para casos donde se necesite.
 - Desventaja: Sistemas de fail-save donde si se detectan accesos no autorizados a los servidores bota el sistema para evitar que puedan seguir inspeccionandolos
- Confiabilidad – Escalabilidad:
 - Ventaja: Sistema con replicaciones idénticas para siempre entregar el mismo dato bajo una misma request.
 - Desventaja: Sistema distribuido con hartos servicios que deben actualizar una misma base de datos. Pueden haber Race conditions.

2. Pregunta UML 1 (29 pts)

Actualmente, los Large Language Models o LLMs predominan en masa en la oferta de AIs comerciales. Estas AIs tienen grandes capacidades para responder a diversos *prompts*, los cuales dictan la información u operaciones que requiere un usuario para satisfacer sus necesidades de datos. Incluso más, varios de estos LLMs proporcionan funcionalidad adicional en forma de tratamiento de información de otros tipos de fuentes (multi-modal) donde se pueden ingresar PDFs, fotos, archivos de datos para dar respuestas de forma aún más fácil a sus usuarios.

Sin embargo, la cantidad no necesariamente implica calidad. No todos estos modelos responden igual y todos tienden a cometer cierto grado de error en su funcionamiento, ya sea proporcionando respuestas incompletas o derechamente erradas (alucinaciones). Estos errores degradan la experiencia del usuario y marcan diferencias sustanciales entre *vendors*.

Para abordar esta situación, LegitQA le encarga generar un sistema capaz de hacer benchmarks semi-automatizados de todos los modelos disponibles de AI en el mercado, con el fin de mostrar los resultados en una plataforma abierta y eficiente. Para hacer este benchmarking, cada *vendor* de AI debe registrar su LLM mediante un identificador de AI (arbitrario), un método de integración (REST o Model Context Protocol) con un schema compatible, una cuenta de servicio y una *API key*. Este registro es en una plataforma sencilla, por invitación mediante correo electrónico.

Al ingresar su AI, cada vendor podrá suscribirse a diversos *challenges*, que son métodos de evaluación del output producido por estas AIs en base a criterios específicos. Estos challenges están clasificados en dos tipos

- Fully-automated: Estos *challenges* están formados por programas previamente proporcionados de forma completamente automática, los cuales pueden enviar prompts de texto y/o multimedia (voz, audio, video) usando la API proporcionada por el vendor y que esperan una determinada respuesta, la cual puede ser evaluada por un pequeño programa validador o alguna AI de referencia propiedad de LegitQA. Estos programas se van actualizando con el tiempo en repositorios Git de origen a medida que van mejorando los *challenges*, puesto que las respuestas mejoran con el tiempo (e.g. Livebench)
- Crowd-dependant: A diferencia de los anteriores, estos LLMs están basados en la votación del público especializado. Se toman ciertos usuarios y se les presenta la respuesta de los modelos frente a diversos escenarios preestablecidos, para posteriormente pedirles un rating de la calidad de la respuesta. Estos escenarios preestablecidos son programas de tamaño pequeño que hacen prompts preguardados específicos y reservan el resultado para enseñarlo a los usuarios en una UI donde pueden acceder (e.g. LLM arena). Al igual que los challenges fully-automated, también van recibiendo actualizaciones con el tiempo

Debe diseñar un sistema que administre estos *challenges* a las AIs registradas con el fin de mostrar un Leaderboard público con los puntajes obtenidos por todos estos challenges, además de reportar los resultados a los *vendors* que registraron sus LLMs. Se

espera que el sistema sea capaz de tener los benchmarks lo antes posible para evitar competidores en este mismo ámbito y que use

Puntos importantes para su conveniencia

- La AI de referencia es un módulo black-box de capacidad limitada con dos interfaces, una de entrada que recibe un resultado de un test dado y una de salida que da un score (1 a 100)
- Los blobs usados por los challenges (y cualquier otro blob que generen) pueden guardarse en un object storage para fácil acceso (al estilo S3). Para evitar congestión de red a causa de blobs, son buckets públicos y se recomienda que su sistema solo pase referencias
- Solo cuenta con un subconjunto de infraestructura básica de la nube (VMs, S3, VPC, API gateway, CDN, ECR, ECS, Fargate o sus equivalentes en GCP o Azure) además de un IdP a elección.
- Recuerde que un diagrama de componentes es distinto a un diagrama de despliegue
- Los *vendors* ofrecen llamadas a sus APIs bajo cierto rate limiting, variable por cada proveedor

2.a) (2 pts) Los challenges podrían ser decenas o cientos. ¿Cómo podría promocionarse la mantenibilidad y escalabilidad rápida de un sistema con tantos programas? Proponga 3 tácticas diferentes y apropiadas. Justifique su uso

Respuesta: Definir las 3 tácticas correcta con su justificación dará el puntaje completo, en caso de falla en alguno se hace el descuento proporcional

2.b) (2 pts) Proponga un sistema de rate limiting y quota enforcement por vendor, para no explotar por los aires los rate limits. Investigue el concepto de Circuit breaker y vea cómo aplicarlo

Respuesta: Se espera usar correctamente los conceptos y proponer un formato en donde se controle la cantidad de pruebas que puede hacer cada IA en un tiempo determinado.

Se da 0.5 puntos cada uno por: usar y definir correctamente cada concepto, proponer el sistema.

Circuit breaker: Funcionalidad de detener los procesos y evitar que se sigan generando nuevos cuando ocurre algún tipo de evento. Debe relacionarlo con evitar que se caiga y proponer ponerlo frente a los componentes de vendor, aunque puede no estar en el diagrama

Quota enforcement: Limitar acceso a un cliente en base a un límite definido numérico

Rate Limit: Límite máximo de accesos a clientes por una cantidad de tiempo

2.c) (4 ptos) Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

Respuesta:

Aplican directamente:

- *Integrabilidad*
- *Usabilidad*
- *Escalabilidad*
- *Mantenibilidad*
- *Confiabilidad*
- *Otro bien justificado*

No aplican:

- *Portabilidad*
- *Soporte*

2.d) (5 ptos) Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

Respuesta:

Aplican directamente:

- *EDA*
- *Pub/Sub*
- *En Capas*
- *Otro bien justificado*

No aplican:

- *Pizarra*
- *Basado en reglas*
- *Main y Subrutinas*
- *Código Móvil*

2.e) (12 ptos) Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Incluya anotaciones explicando funcionamiento y supuestos, así como nombres de interfaces

Respuesta: Si no es un diagrama de componentes UML no se corrige. Como puede haber muchas soluciones al problema, se analizará en base a descuentos:

- (-3 c/u) No usan los estilos o el componente que envía los datos que justificaron previamente
- (-2 c/u) Falta un requisito funcional
- (-1 c/u) Interfaces mal implementadas
- (-1 c/u) Componentes desagregados (sin conectores)
- (-2) No hay nombres en interfaces

2.f) (4 ptos) Identifique 2 trade-off originados de la solución de su diagrama. Comente como las solucionaría y las implicancias de estas soluciones. No se corregirán aquellas que apunten al formato de cómo se diagramó o si falto explotar algún componente

Respuesta: Se esperan trade-off asociados a su solución, puede ser por una decisión de implementar un estilo u herramienta o por el mismo diseño de su sistema. Sobre la distribución de puntaje:

- (1pto) Identificar un trade-off correctamente
- (1pto) Identificar correctamente una posible solución aplicable a su sistema

3. Pregunta UML 2 (27 pts)

Dado el creciente aumento de la industria de los videojuegos para consolas, muchos desarrolladores jóvenes quieren involucrarse en este sin tener mayor experiencia, generándoles problemas al momento de **optimizar sus juegos para las distintas plataformas usando**, muchas veces, computadores y máquinas que no son las mejores.

Con esto en mente, la empresa LegitGaming encontró un modelo de negocio para estas personas aprovechando las capacidades actuales de **transmisión de datos en la nube**. Prometen un sistema en donde estas personas puedan **programar sus juegos a su gusto sin preocuparse de la eficiencia** para que luego sus sistemas online puedan compactarlos y optimizarlos usando sus propios recursos de la nube.

El sistema de LegitGaming deberá tener un **Frontend Web** en donde los distintos usuarios desarrolladores **indicarán a que consola quieren optimizar su juego y subirán sus archivos de juegos programados** en el motor gráfico que más les acomode (Unity, Unreal Engine, etc), para que la plataforma les indique que **se ha subido correctamente** y que **estará siendo procesado de forma asíncrona**. Acá un servicio inicial deberá guardar los archivos crudos para que el resto del sistema pueda obtenerlos e inicia el procesamiento asíncrono.

Para el procesamiento tendrán **2 servicios** que cada uno se encargará de una optimización en particular. Primeramente, tendrán un servicio encargado de **optimizar todas las imágenes, sprites y sonidos del sistema para que calcen dentro del formato específico** (por ejemplo, reducir las imágenes a mapeos de 1080x1080 pixeles) usando una **base de datos** con la **información de cada regla**, que deben seguir para cada consola, que generarán distintos **filtros para cada archivo**. Estos **archivos se deberán guardar y marcar en una base de datos** que ya se pueden descargar por parte del cliente.

Luego de esto, tendrán otro servicio que se encargará de **optimizar el código para que no use tantos recursos de las consolas**. Para esto, deberán tomar los archivos de código y **filtrarlos hasta para encontrar las partes más demandantes**, y luego deberán usar una **IA especializada en estas optimizaciones** que desarrollo LegitGaming que les entregará los fragmentos optimizados que deberán unir al código base. Luego de esto deberán volver a **guardar los archivos para permitir la descarga** de los usuarios e **indicar que ya está listo** para ser descargado.



Finalmente, una vez que se **termine de procesar todo**, su sistema deberá **actualizarse para indicarle al usuario que los puede descargar**.

3.a) (2 pts) Como muchos de estos juegos serán tendrán derechos de autor de sus creadores, les indican que deberán mantener un nivel de seguridad en los accesos a la APIs para descargar

y subir los archivos. De al menos 2 soluciones que puede aplicar en su sistema para asegurar esto.

Respuesta: Las soluciones indicadas deben de poder abarcar conceptos básicos del atributo de calidad seguridad, en particular sobre autorización de usuarios. Guiándose por lo realizado en su E1, puede ser autenticación y autorización de usuarios en la plataforma.

3.b) (4 pts) Indique (mínimo 3, pero pueden ser más) atributos de calidad vistos en clases que Ud. considere son fundamentales en este problema. Enumérelos en orden de prioridad y justifique brevemente.

Respuesta:

Aplican directamente:

- Escalabilidad
- Confiabilidad
- Disponibilidad
- Seguridad
- Flexibilidad: Se permite argumentar porque pueden ir saliendo nuevas consolas
- Otro bien justificado

No aplican:

- Usabilidad: Los usuarios solo interactúan para subir o descargar sus archivos
- Eficiencia: El sistema supe la eficiencia de los usuarios, pero el sistema en si no lo necesita
- Portabilidad
- Mantenibilidad

3.c) (5 pts) Indique (mínimo 2, pero pueden ser más) los estilos arquitectónicos vistos en clases que va a emplear para solucionar el problema. Indique qué aspecto del problema ataca cada estilo (responsabilidad).

Respuesta:

Aplican directamente:

- EDA
- Pub/Sub
- Pipe & Filter
- Basado en reglas
- Microservicios
- Otro bien justificado

No aplican:

- Peer to Peer
- Pizarra

- *Cliente Servidor: Es muy general para el sistema, y solo podría darse por un conocimiento técnico*
- *Main y Subrutinas*

3.d) (12 ptos) Confeccione un diagrama de componentes UML indicando los componentes arquitectónicos y sus relaciones, además de sus estilos especificados previamente, tal que reflejen una solución al problema planteado. Añada tanto detalle como haga falta para que quede claro cómo soluciona el problema planteado. Incluya anotaciones explicando funcionamiento y supuestos, así como nombres de interfaces

Respuesta: Si no es un diagrama de componentes UML no se corrige. Como puede haber muchas soluciones al problema, se analizará en base a descuentos:

- (-3 c/u) No usan los estilos o el componente que envía los datos que justificaron previamente
- (-2 c/u) Falta un requisito funcional
- (-1 c/u) Interfaces mal implementadas
- (-1 c/u) Componentes desagregados (sin conectores)
- (-2) No hay nombres en interfaces

Diagrama de referencia: [I1_P2 Referencia](#)

3.e) (4 ptos) Identifique 2 trade-off originados de la solución de su diagrama. Comente como las solucionaría y las implicancias de estas soluciones. No se corregirán aquellas que apunten al formato de cómo se diagramó o si faltó explotar algún componente

Respuesta: Se esperan trade-off asociados a su solución, puede ser por una decisión de implementar un estilo u herramienta o por el mismo diseño de su sistema. Sobre la distribución de puntaje:

- (1pto) Identificar un trade-off correctamente
- (1pto) Identificar correctamente una posible solución aplicable a su sistema